

	Course Name: Advanced Web Technology	EXPERIMENT NO. 1	
	Course Code: 20CP314P Faculty: Komal Singh	Branch: CSE	Semester: VI
Submitted by: Harsh Shah Roll no: 21BCP359			

Objective: Building a Simple Web Server in Node.js

Experiment: Implement the NodeJS Programs.

Q.1 Write Node JS programs for following modules:

a. http

```
const http = require('http');
const server = http.createServer((req, res) =>
{ res.writeHead(200, { 'Content-Type': 'text/plain' });
res.end('Hello World!\n');
});
server.listen(3000, 'localhost', () =>
{ console.log('Server running at
http://localhost:3000/')
})
```

Output:-

```
Hello World!
```

```
[nodemon] restarting due to changes...
[nodemon] starting `node temp.js`
Server running at http://localhost:3000/
```

b. filesystem

```
const fs = require('fs');
const fileName = 'example.txt';
const fileContent = 'This is a sample text file.';
fs.writeFile(fileName, fileContent, (err) => {
  if (err) throw err;
  console.log(`${fileName} has been created.`);
});
fs.readFile(fileName, 'utf8', (err, data) => {
  if (err) throw err;
  console.log(`Contents of ${fileName}:`);
  console.log(data);
});
```

Output:-

```
example.txt has been created.
Contents of example.txt:
This is a sample text file.
```

c. URL

```
const url = require('url');
const urlString =
'https://www.example.com/path/to/resource?query1=value1&query2=value2';
const parsedUrl = url.parse(urlString, true);
console.log('Parsed URL:');
console.log(parsedUrl);
```

Output:-

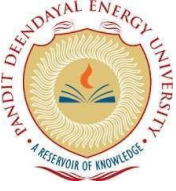
```
Parsed URL:
Url {
  protocol: 'https:',
  slashes: true,
  auth: null,
  host: 'www.example.com',
  port: null,
  hostname: 'www.example.com',
  hash: null,
  search: '?query1=value1&query2=value2',
  query: [Object: null prototype] { query1: 'value1', query2: 'value2' },
  pathname: '/path/to/resource',
  path: '/path/to/resource?query1=value1&query2=value2',
  href: 'https://www.example.com/path/to/resource?query1=value1&query2=value2'
}
```

d. Event

```
const EventEmitter = require('events');
class MyEmitter extends EventEmitter {}
const myEmitter = new MyEmitter();
myEmitter.on('event', () => {
  console.log('Event emitted!');
});
myEmitter.emit('event');
```

Output:-

```
[nodemon] starting node temp.js
Event emitted!
[nodemon] clean exit - waiting for changes
```

	Course Name: Advanced Web Technology	EXPERIMENT NO. 2	
	Course Code: 20CP314P Faculty: Komal Singh	Branch: CSE	Semester: VI
Submitted by: Harsh Shah Roll no: 21BCP359			

Objective: Building a Web Server with Express.js

Experiment 2: Implement with routing methods (GET and POST) and also implement one built-in middleware, configurable middleware and one third party middleware (Cookie-Parser).

Hint: You can also use the function validate cookie in third party middleware.

App.js

```
const express = require('express');
const cookieParser = require('cookie-parser'); const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: true })); app.use(cookieParser());

// Configurable middleware
const myMiddleware = (config)
=> { return (req, res, next)
=> {
  console.log(`Configurable Middleware:
    ${config}`); next();
};
};

// GET route
app.get('/', (req, res) =>
{ res.send('Hello, this is a GET
  route!');
});

// POST route with configurable middleware and cookie handling
app.post('/', myMiddleware('I am configurable middleware'), (req, res) =>
{
  console.log('Request Body: ', req.body);
});
```

```

// Retrieve and validate the cookie
const myCookie =
req.cookies.myCookie; if (myCookie
=== 'cookieValue') {
    res.send('Hello, this is a POST route! Cookie is valid.');
```

```

} else {
    res.send('Hello, this is a POST route! Cookie is invalid.');
```

```

}
});
const PORT = 3000;
app.listen(PORT, ()
=> {
    console.log(`Server is running on http://localhost:${PORT}`);

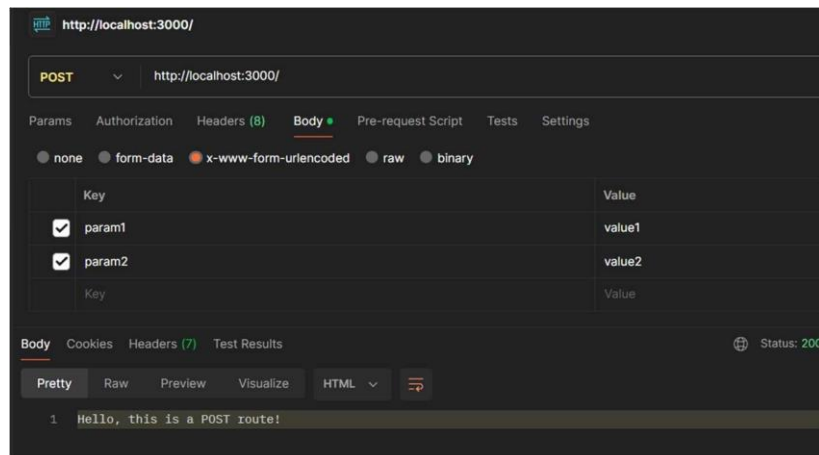
```

Output :-

Get Request :-



Post Request With Cokkie-parser :-



	Course Name: Advanced Web Technology	EXPERIMENT NO. 3	
	Course Code: 20CP314P Faculty: Komal Singh	Branch: CSE	Semester: VI
(To be filled by Student) Submitted by: Harsh Shah Roll no: 21BCP359			

Objective: Setting up a MongoDB Database (Connecting MongoDB to your application)

Experiment 3: Create a JavaScript file with MongoDB queries for operations such as insert, update, and delete while also establishing a connection to the MongoDB database.

Hint: Ensure that your MongoDB server is running and accessible at localhost:27017 or replace it with the appropriate connection string if it's hosted elsewhere.

Code:-

```
const mongoose = require('mongoose');
```

```
const validator = require('validator');
```

```
const uri =
```

```
'mongodb+srv://harshshah:harsh@cluster0.imhkvus.mongodb.net/harsh?retryWrites=true&w=majority';
```

```
mongoose.connect(uri, { useNewUrlParser: true, useUnifiedTopology: true });
```

```
const friendSchema = new mongoose.Schema({
```

```
  name: String,
```

```
  age: Number,
```

```
  email: {
```

```
    type: String,
```

```
    required: true,
```

```
    validate: {
```

```
      validator: validator.isEmail,
```

```
        message: '{VALUE} is not a valid email'
    }
},
active: Boolean
});
```

```
const Friend = mongoose.model("Harsh_W", friendSchema);
const inst_v = [
    { name: "HarshShah", age: 20, email: "HarshShah@gmail.com", active: true },
    { name: "Karan", age: 45, email: "Karan@example.com", active: true } // Added comma
    after age property
];
```

```
async function main() {
    try {
        // Insert documents
        await Friend.insertMany(inst_v);

        // Update documents
        await Friend.updateMany({ name: "Karan" }, { $set: { age: 45 } });

        // Delete documents
        await Friend.deleteOne({ name: "Karan" });

        // Read documents
        const read = await Friend.find({ name: "HarshShah" });
        console.log(read);

    } catch (err) {
        console.log(err);
    } finally {
```

```
        mongoose.connection.close();
    }
}
main()
```

Output:-

1.) INSERT

```
[
  {
    _id: ObjectId("65c6e1b53ea5e8f810f5e3d1"),
    name: "HarshShah",
    email: "HarshShah@gmail.com",
    age: 20,
    active: true
  },
  {
    _id: ObjectId("65c6e1b53ea5e8f810f5e3d2"),
    name: "Karan",
    email: "Karan@example.com",
    age: 45,
    active: true
  }
]
```

2.) UPDATE AND DELETE

```
[
  {
    _id: ObjectId("65c6e1b53ea5e8f810f5e3d1"),
    name: "HarshShah",
```

```
    email: "HarshShah@gmail.com",  
    age: 20,  
    active: true  
  }  
]
```

